Software Requirements Specification (SRS)

SmartCart Distributed E-Commerce Platform

Document Identifier: SRS-SMARTCART-001
Document Version: 1.0.0 (Baseline Specification)
Lifecycle Phase: Requirements Analysis & Specification Phase (Pre-Implementation Baseline)
Specification Standard: ISO/IEC/IEEE 29148:2018 / IEEE Std 830-1998
Date of Approval: October 16, 2026
Status: Approved Engineering Baseline
Architecture Classification: Polyglot Microservices Architecture
Engineering Team: Group 03 (TeamID 03)
Target Repository: https://github.com/Ayush17314/SmartCart
Project Deliverables Drive: https://drive.google.com/drive/folders/1q9IY8r2_aUCeogiUsT0tpODwt3hF6PHL?usp=sharing

Executive Summary & Engineering Foreword

This document represents the formal **Software Requirements Specification (SRS)** for the **SmartCart Distributed E-Commerce Platform**, authored during the **Requirements Engineering Phase** of the Software Development Life Cycle (SDLC).

In accordance with software engineering principles, this specification establishes the binding functional, behavioral, architectural, and quality attribute baseline **prior to system implementation**. It defines the contractual agreements between product stakeholders, system architects, software developers, and quality assurance engineers. All subsequent design models, API schemas, service implementations, and test suites shall be validated against the verifiable requirements specified herein.

Table of Contents

1. Introduction
   - 1.1 Purpose
   - 1.2 Document Conventions & RFC 2119 Terms
   - 1.3 Intended Audience & Reading Suggestions
   - 1.4 Product Scope & Boundaries
   - 1.5 Applicable References & Industry Standards
2. Overall Description
   - 2.1 Product Perspective & Context Diagram
   - 2.2 Summary of Major Product Functions
   - 2.3 User Classes and Operational Personas (RBAC Hierarchy)
   - 2.4 Operating Environment & Planned Port Allocations
   - 2.5 Design, Technical and Implementation Constraints

- 2.6 Operational Assumptions and System Dependencies
- 2.7 Phased Implementation Roadmap (Engineering Milestones 1–6)

3. System Architecture & Structural Decomposition
   - 3.1 Architectural Decomposition & Domain Isolation
   - 3.2 Service Topology & Infrastructure Blueprint
   - 3.3 Inter-Service Protocol & Security Model
4. External Interface Requirements
   - 4.1 User Interfaces (UI/UX)
   - 4.2 Hardware & Network Interfaces
   - 4.3 External Software & Partner Service Interfaces
   - 4.4 Communications Interfaces & Content Negotiation
5. Specific Functional Requirements
   - 5.1 API Gateway Service (`GW`)
   - 5.2 Authentication & Identity Service (`AUTH`)
   - 5.3 Product Catalog & Inventory Service (`PROD`)
   - 5.4 Shopping Cart Service (`CART`)
   - 5.5 Wishlist Service (`WISH`)
   - 5.6 Order Orchestration & Promotions Service (`ORD`)
   - 5.7 Payment & Financial Reconciliation Service (`PAY`)
   - 5.8 Notification & Communication Service (`NOTIF`)
   - 5.9 Storefront & Management Web Client (`UI`)
6. Non-Functional Requirements (NFRs)
   - 6.1 Security & Cryptographic Specifications
   - 6.2 Reliability, Fault Tolerance & Resilience
   - 6.3 Performance & Caching Attributes
   - 6.4 Observability, Logging & Auditability
   - 6.5 Maintainability, Portability & Configuration Standards
7. Data Requirements & Persistence Models
   - 7.1 Database Isolation & Data Ownership Rules
   - 7.2 Relational Data Schema (MySQL `smartcart`)
   - 7.3 Document Data Schemas (MongoDB)
   - 7.4 Data Integrity, Optimistic Concurrency & Lifecycle Rules
8. Verification, Validation & Acceptance Criteria
   - 8.1 Verification Methodology
   - 8.2 Requirements Traceability Matrix (RTM)
   - 8.3 Standard HTTP Status Code Baseline
   - 8.4 RFC 7807 Problem Details Error Catalogue
9. Appendix & Sign-Off
   - 9.1 Glossary & Acronyms
   - 9.2 Engineering Sign-Off Record

---

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to establish the authoritative, complete, and verifiable technical requirements for the **SmartCart Distributed E-Commerce Platform**.

This document serves as the primary artifact of the **Software Requirements Engineering Phase**. It is authored **prior to the construction of the software system** to:
1. Formulate a shared technical contract between product owners, system architects, developers, and testers.
2. Formally define functional capabilities, microservice boundaries, interface protocols, and data ownership models.
3. Establish non-functional criteria regarding security, performance, resilience, fault tolerance, and standards compliance.
4. Provide the verifiable baseline for software design specifications, test plan generation, and acceptance verification.

1.2 Document Conventions & RFC 2119 Terms

The requirement statements in this document strictly adhere to **BCP 14** / **RFC 2119** and **RFC 8174** key terminology:
- **SHALL / MUST / REQUIRED:** An absolute, binding requirement of the specification.
- **SHALL NOT / MUST NOT:** An absolute prohibition of the specification.
- **SHOULD / RECOMMENDED:** A valid reason may exist in particular circumstances to ignore a particular item, but the full implications must be understood and carefully weighed.
- **MAY / OPTIONAL:** An item is truly discretionary.

Identifier Conventions:
- **Functional Requirements:** Formatted as `FR-[SERVICE]-[NUMBER]` (e.g., `FR-GW-01`, `FR-ORD-02`).
- **Non-Functional Requirements:** Formatted as `NFR-[CATEGORY]-[NUMBER]` (e.g., `NFR-SEC-01`, `NFR-REL-02`).
- **Data Entities:** Formatted in capitalized camelCase or singular nouns (e.g., `Order`, `Product`, `IdempotencyRecord`).

1.3 Intended Audience & Reading Suggestions

- **System Architects:** Review Sections 2, 3, and 7 to evaluate component boundaries, isolation models, and persistence architecture.
- **Backend Developers:** Review Sections 4, 5, 6, and 7 for API contracts, domain validation logic, and error-handling requirements.
- **Frontend Engineers:** Review Sections 4, 5.9, 6.3, and 8.4 for API payload structures, session flows, and Problem Details parsing.
- **Quality Assurance & Verification Teams:** Review Section 8 (Verification and Requirements Traceability Matrix) to construct black-box, white-box, and integration test plans.
- **Academic Evaluators & Project Reviewers:** Review Section 2.7 and Section 8.2 for alignment with Web Services engineering assignments (Assignments 1 through 6).

1.4 Product Scope & Boundaries

SmartCart is an enterprise-grade, Amazon-inspired e-commerce platform engineered with a **polyglot microservices backend** and a high-performance **Next.js 15 Web application frontend**.

In-Scope Capabilities:

- Unified catalog search, categorization, dynamic brand filtering, and cursor-based pagination.
- Multi-tier Role-Based Access Control (RBAC) supporting Customer, Support, Inventory Staff, Admin, and Super Admin personas.
- Persistent shopping cart management and saved-for-later wishlist operations.
- Distributed order checkout orchestration with transactional stock reservation and compensatory rollbacks.
- Multi-channel payment processing supporting online digital payments (UPI, Credit/Debit cards via Razorpay API) and offline Cash on Delivery (COD).
- Asynchronous payment verification via HMAC-SHA256 signature validation and provider webhooks.

- Order state machine management, return authorization workflows, and automated financial refunds.
- Transactional notification dispatch via cloud email gateways with decoupled mock testing fallbacks.
- HTTP-level robustness: RFC 7807/9457 Problem Details, dynamic ETag caching, optimistic concurrency control (`If-Match`), content negotiation (`Accept`), sliding-window rate limiting, circuit breaking, and durable idempotency.

Out-of-Scope Capabilities:

- Direct physical warehouse robotics integration.
- Native mobile client compilation (iOS/Android native binaries); access is provided via responsive progressive web interfaces.
- Legacy SOAP protocol gateways (the architecture strictly mandates lightweight RESTful JSON and GraphQL interfaces).

1.5 Applicable References & Industry Standards

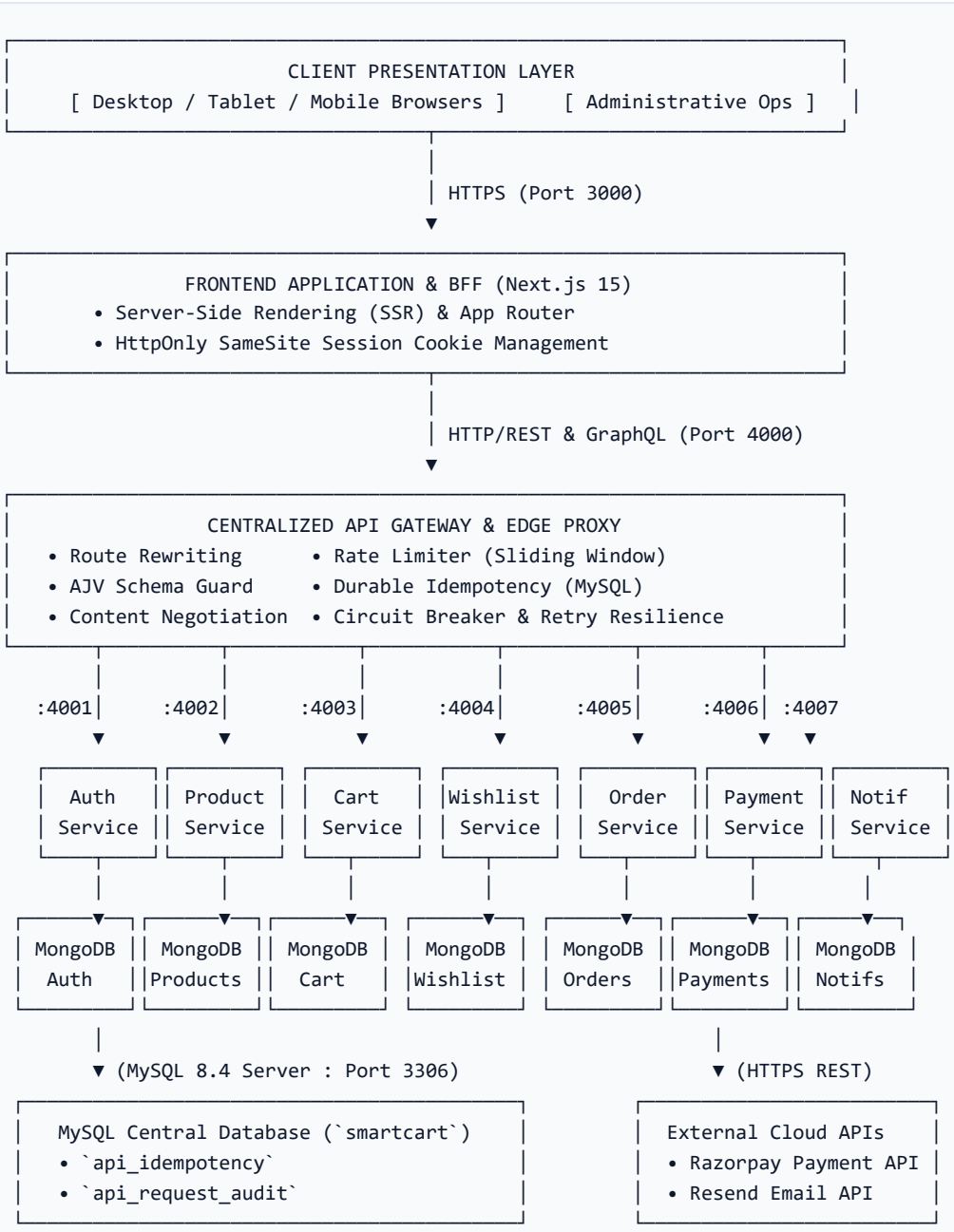
1. **ISO/IEC/IEEE 29148:2018:** Systems and software engineering — Life cycle processes — Requirements engineering.
2. **IEEE Std 830-1998:** IEEE Recommended Practice for Software Requirements Specifications.
3. **RFC 7230–7235:** Hypertext Transfer Protocol (HTTP/1.1) Message Syntax, Semantics, Conditional Requests, and Caching.
4. **RFC 7807 / RFC 9457:** Problem Details for HTTP APIs (`application/problem+json`).
5. **RFC 7519:** JSON Web Token (JWT) Architecture and Security Guidelines.
6. **RFC 5988:** Web Linking (Pagination `Link` headers).
7. **OpenAPI Specification (OAS) v3.0.3:** Standardized Machine-Readable RESTful API Contracts.
8. **NIST SP 800-63B:** Digital Identity Guidelines — Authentication and Lifecycle Management.

---

2. Overall Description

2.1 Product Perspective & Context Diagram

The SmartCart platform is planned as a decentralized, service-oriented architecture. Individual microservices shall strictly own their domain persistence models and business logic without cross-service database coupling. All cross-domain interactions shall be mediated via HTTP/REST endpoints, asynchronous webhooks, and a centralized reverse-proxy API Gateway.



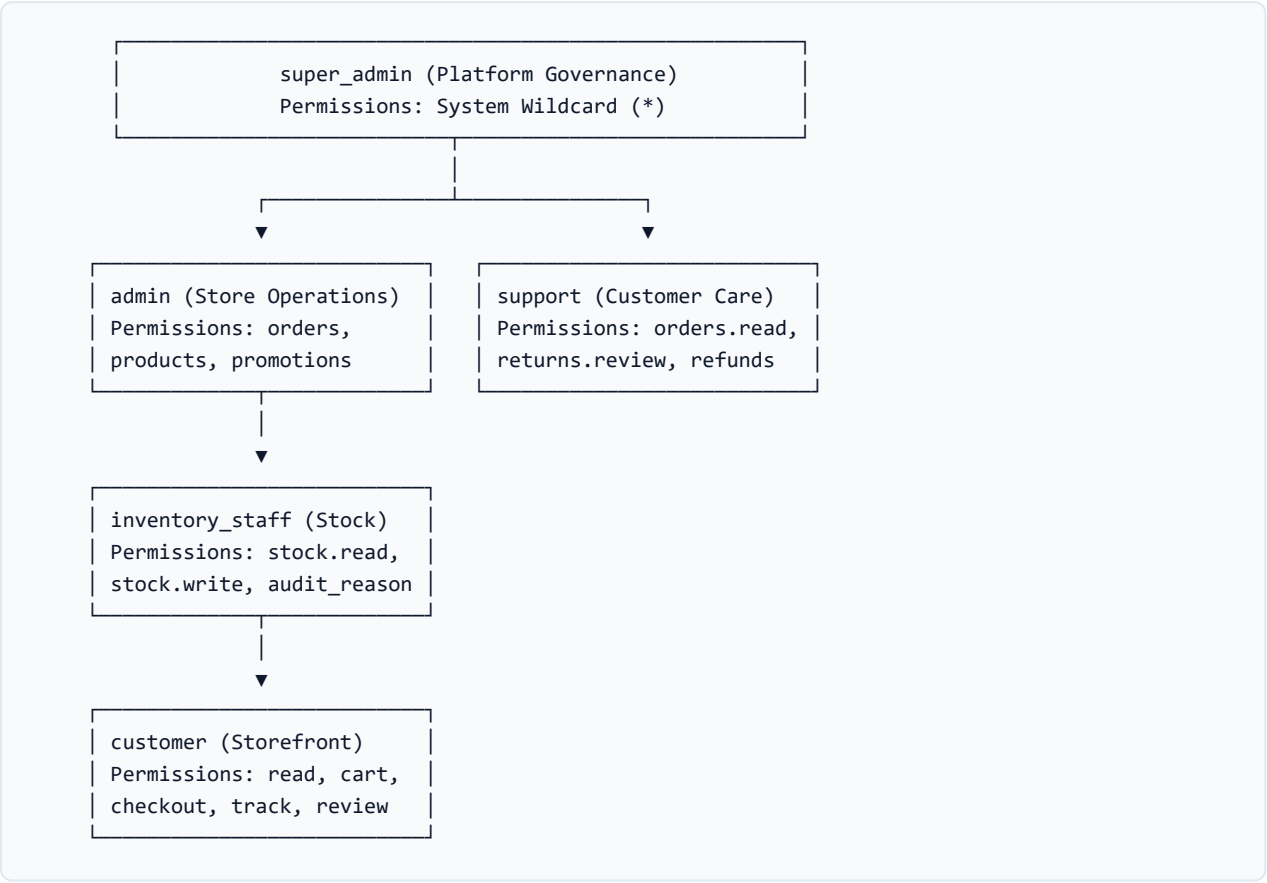
2.2 Summary of Major Product Functions

The planned software system shall deliver seven core functional pillars:
1. **Catalog & Discovery:** Rich catalog navigation, full-text product search, category hierarchies, conditional caching via HTTP ETags, and cursor-based pagination.
2. **Identity & Access Governance:** Multi-portal authentication, cryptographically hashed passwords, role-based authorization, session revocation, and staff invitation lifecycles.
3. **Cart & Wishlist Sessions:** Persistent server-side cart calculation, line-item quantity management, and seamless move-to-cart wishlist operations.
4. **Checkout & Order Orchestration:** Distributed saga orchestration validating cart state, reserving inventory, evaluating coupon discounts, and invoking payment creation with compensatory rollback logic.
5. **Payment Processing & Reconciliation:** Multi-channel payments via Razorpay (UPI, Credit/Debit cards) and Cash on Delivery (COD), cryptographically verified via HMAC-SHA256 signatures and webhook processing.
6. **Order State Lifecycle & Returns:** Strict state machine enforcement (`PENDING` $\rightarrow$ `CONFIRMED` $\rightarrow$ `SHIPPED` $\rightarrow$ `DELIVERED`), delivery tracking, 30-day return evaluation, and financial refunds.

7. **Transactional Notifications:** Automated email delivery for account security, order confirmation, shipping updates, and administrative challenge codes.

2.3 User Classes and Operational Personas (RBAC Hierarchy)

The system requirements specify five discrete user classes, each with distinct operational characteristics and permissions:



User Persona	Technical Role Identifier	Access Gateway & Portal	Target Responsibilities & System Privileges
Store Customer	`customer`	Storefront (`/login` , `/cart` , `/checkout`)	Search catalog; maintain cart/wishlist; place orders; complete digital payments; track shipments; submit reviews; initiate return requests within 30 days of delivery.
Customer Support	`support`	Staff Portal (`/admin/login`)	Read-only view of customer orders and accounts; evaluate return requests; approve or reject returns with mandatory justification; queue refund requests.
Inventory Staff	`inventory_staff`	Staff Portal (`/admin/login`)	Create catalog listings; update stock counts; mandatory entry of audit explanation for every stock increment/decrement; review low-stock thresholds.
Store Admin	`admin`	Admin Management Suite (`/admin`)	Manage order lifecycle states (`CONFIRMED` $\rightarrow$ `SHIPPED` $\rightarrow$ `DELIVERED`); assign carrier tracking numbers; create/expire coupon discounts; moderate customer reviews.

User Persona	Technical Role Identifier	Access Gateway & Portal	Target Responsibilities & System Privileges
Super Admin	`super_admin`	Super Admin Console (`/super-admin`)	Complete system wildcard (`*`); 2-step verification code challenge; provision staff invitations; authorize financial refunds via payment gateway; configure store taxes and shipping fees.

2.4 Operating Environment & Planned Port Allocations

The system shall execute within containerized Linux environments (Ubuntu 22.04 LTS / Alpine Linux) or cross-platform developer workstations (Windows 10/11, macOS Darwin).

Subsystem Component	Planned Port	Runtime Environment	Backing Data Store	Core Functional Responsibility
Storefront Web App	`3000`	Next.js 15, React 19, Node.js v20+	Client Cookie / Session Cache	Storefront UI, Admin Suites, Staff Portal, BFF Proxy
API Gateway	`4000`	Node.js, Express, Helmet, AJV	MySQL 8.4 (`smartcart`)	Reverse Proxy, Rate Limiting, Idempotency, RFC 7807, OpenAPI
Auth Service	`4001`	Node.js, Express, Mongoose	MongoDB (`smartcart_auth`)	User Identity, RBAC Tokens, Password Hashing, Staff Invites
Product Service	`4002`	Node.js, Express, Mongoose	MongoDB (`smartcart_products`)	Catalog Search, Categories, Inventory History, Reviews
Cart Service	`4003`	Node.js, Express, Mongoose	MongoDB (`smartcart_cart`)	User Carts, Server-Side Pricing, Quantity Rules
Wishlist Service	`4004`	Node.js, Express, Mongoose	MongoDB (`smartcart_wishlist`)	Saved Items, Move-to-Cart Orchestration
Order Service	`4005`	Node.js, Express, Mongoose	MongoDB (`smartcart_orders`)	Checkout Saga, State Machine, Coupons, Shipment Tracking
Payment Service	`4006`	Node.js, Express, Mongoose	MongoDB (`smartcart_payments`)	Razorpay Orders, HMAC Verification, COD, Refunds
Notification Service	`4007`	Node.js, Express, Mongoose	MongoDB (`smartcart_notifications`)	Transactional Emails, Provider Fallbacks, Audit Log
Relational Database	`3306`	MySQL 8.4 Community	InnoDB Storage Engine	Distributed Idempotency Store, Gateway Request Audits
Document Database	`27017`	MongoDB Community 7.0 / Atlas	WiredTiger Engine	Microservice Domain Data Persistence

2.5 Design, Technical and Implementation Constraints

1. **Zero Shared Databases (Microservice Autonomy):** Each microservice SHALL strictly maintain its own dedicated database schema. No microservice shall query or modify another microservice's database directly.
2. **Backend-for-Frontend (BFF) Token Secrecy:** Browser clients SHALL NOT receive raw JWT bearer tokens. The Next.js BFF server shall maintain JWT credentials within secure, `HttpOnly`, `SameSite=Lax` cookies.
3. **Stateless API Gateway:** The API Gateway SHALL NOT store session state in server memory; idempotency and rate limiting states must be externalized to MySQL or Redis for horizontal scalability.
4. **RFC 7807/9457 Uniform Error Handling:** All non-2xx HTTP responses across all services SHALL conform to the RFC 7807 Problem Details JSON format (`application/problem+json`).
5. **No Legacy SOAP:** The system requirements strictly mandate lightweight, web-native RESTful JSON and GraphQL interfaces, eliminating heavy XML/SOAP protocol overhead.

2.6 Operational Assumptions and System Dependencies

1. **Cloud Service Availability:** The system assumes constant network access to the Razorpay API (`https://api.razorpay.com/v1`) and Resend Email API (`https://api.resend.com/emails`).
2. **Docker Network Topology:** Internal microservices shall be interconnected via a bridged Docker network (`smartcart-network`) allowing DNS resolution by service name.
3. **Client Modernity:** Web clients are assumed to support modern ECMAScript standards, HTML5 web storage, and CSS Flexbox/Grid layouts.

2.7 Phased Implementation Roadmap (Engineering Milestones 1–6)

To ensure systematic development, testing, and academic verification, the implementation shall proceed across six structured milestones:
- **Milestone 1 (Foundations & Raw HTTP):** Repository setup, service directory scaffolding, environment management, and foundational HTTP handling.
- **Milestone 2 (Domain Decomposition & Contracts):** Microservice boundary definitions, isolated database schema creation, cross-service HTTP orchestration, and checkout saga.
- **Milestone 3 (External Partner Integration):** Razorpay payment gateway integration, cryptographic HMAC-SHA256 signature verification, webhook listeners, and refund flows.
- **Milestone 4 (REST Design, Idempotency & Resilience):** Plural resource naming, OpenAPI 3.0 documentation, durable `Idempotency-Key` tracking, circuit breakers, and safe retries.
- **Milestone 5 (Advanced HTTP Semantics & Caching):** Dynamic ETag generation (304), optimistic concurrency (`If-Match` /412), `Accept` header content negotiation (406), and sliding-window rate limiting (429).
- **Milestone 6 (Honest Error Standardization & Robustness):** Comprehensive RFC 7807 Problem Details deployment, multi-field validation error arrays (400/422), canonical `Location` headers on 201 Created, and masked 500 error boundaries.

---

3. System Architecture & Structural Decomposition

3.1 Architectural Decomposition & Domain Isolation

The platform architecture is specified according to the **Domain-Driven Design (DDD)** bounded context pattern:

DOMAIN BOUNDED CONTEXTS	
Identity Context	Auth Service
Catalog & Inventory Context	Product Catalog Service
Shopping Experience Context	Cart Service, Wishlist Service
Sales & Fulfillment Context	Order Orchestration Service
Billing & Treasury Context	Payment & Reconciliation Service
Communications Context	Notification Service
Edge & Security Context	API Gateway, Reverse Proxy

3.2 Service Topology & Infrastructure Blueprint

- All external client ingress shall terminate at the **Next.js 15 Web Application** (Port 3000) or directly at the **API Gateway** (Port 4000).
- Upstream microservices (Ports 4001–4007) shall reside inside an isolated private subnet or internal Docker network, inaccessible from public Internet routing.
- The API Gateway acts as the single choke-point enforcing rate limits, request schema validation, idempotency checks, and correlation tracking.

3.3 Inter-Service Protocol & Security Model

1. **Correlation Tracking:** The API Gateway SHALL generate a unique UUID v4 and inject it as header `X-Request-Id` on all proxied requests. Every downstream service shall propagate this ID through internal service hops and log messages.
2. **Internal Secret Tokens:** Microservices performing sensitive cross-service operations (e.g., Auth token exchange, Payment callbacks) SHALL require pre-shared internal secret headers (`X-Auth-Exchange-Secret` , `X-Payment-Service-Secret`).
3. **Timing-Safe Header Comparison:** Downstream services verifying internal secret tokens SHALL utilize constant-time comparison algorithms (`crypto.timingSafeEqual`) to prevent timing attack vulnerabilities.

---

4. External Interface Requirements

4.1 User Interfaces (UI/UX)

- **UI-01 (Responsive Breakpoints):** The storefront and management portals shall be fully responsive across mobile ($\le 640\text{px}$), tablet ($768\text{px} - 1024\text{px}$), and desktop ($\ge 1280\text{px}$) viewports.
- **UI-02 (Accessibility Compliance):** The interface shall comply with **WCAG 2.1 Level AA** standards, providing semantic HTML5 elements, ARIA role labels, and high-contrast color ratios.
- **UI-03 (State Feedback & Skeletons):** Dynamic catalog searches and page transitions shall render shimmer skeleton loaders to prevent layout shifts.

4.2 Hardware & Network Interfaces

- **NET-01 (Protocol Support):** External ingress shall operate over **HTTPS (TLS 1.2 / TLS 1.3)** over TCP Port 443.
- **NET-02 (Internal Bridge):** Inter-service container communications shall operate over TCP/IP over internal port range `4000–4007` .

4.3 External Software & Partner Service Interfaces

- **EXT-01 (Razorpay Payment Gateway API):**
- Order creation: `POST https://api.razorpay.com/v1/orders` (Basic Auth with Key ID and Secret).
- Refund initiation: `POST https://api.razorpay.com/v1/payments/:id/refund`.
- Inbound webhook processing: `POST /api/payments/webhook` with raw request signature validation.
- **EXT-02 (Resend Transactional Email API):**
- Inbound dispatch: `POST https://api.resend.com/emails` (Bearer Token authorization).
- Fallback mode: When `RESEND_API_KEY` is undefined, the Notification Service shall log formatted email payloads to stdout without error.
- **EXT-03 (Google OAuth 2.0 Identity Platform):**
- OIDC discovery and token verification endpoints for single-click customer authentication.

4.4 Communications Interfaces & Content Negotiation

- **COM-01 (Content Negotiation):** The API Gateway shall evaluate the incoming HTTP `Accept` header. Requests explicitly demanding unsupported media representations shall be rejected with HTTP `406 Not Acceptable`. Supported media types:
- `application/json` (Primary default)
- `application/problem+json` (Standard error payload)
- `application/xml` and `text/xml` (Catalog exports)
- **COM-02 (OPTIONS Method Support):** The API Gateway and microservices shall respond to HTTP `OPTIONS` requests with HTTP status `204 No Content` and an explicit `Allow` header declaring supported HTTP verbs.

---

5. Specific Functional Requirements

5.1 API Gateway Service (`GW`)

FR-GW-01: Reverse Proxy & Path Rewriting

- **Objective:** Route client requests to designated microservice ports based on route prefixes.
- **Specification:** The Gateway SHALL accept requests on Port 4000 and proxy them to downstream services. The Gateway SHALL support both `/api/*` and versioned `/v1/api/*` prefixes, stripping `/v1` transparently before forwarding.

FR-GW-02: Sliding-Window Rate Limiting

- **Objective:** Protect platform services from denial-of-service and brute-force attacks.
- **Specification:** The Gateway SHALL enforce a sliding-window rate limit (default 120 requests per 60 seconds per IP/client).
- **Behavior on Violation:** Requests exceeding quota SHALL receive HTTP status `429 Too Many Requests` formatted as RFC 7807 Problem Details.
- **Headers:** The response MUST include `RateLimit-Limit`, `RateLimit-Remaining`, `RateLimit-Reset`, and `Retry-After: <seconds>`.

FR-GW-03: Durable Idempotency Management

- **Objective:** Prevent duplicate order creation, charges, or stock deductions caused by network retries.

- **Specification:** For mutating requests (`POST /api/orders` , `/api/products` , `/api/cart/items` , `/api/payments/create`), the client MUST supply an `Idempotency-Key` header (1–200 characters).
  1. The Gateway SHALL compute a SHA-256 hash of the request body, method, and URL.
  2. If the key exists in MySQL table `api_idempotency` with an identical hash, the Gateway SHALL immediately return the cached status code, headers, and body with header `Idempotency-Replayed: true` .
  3. If the key exists with a differing payload hash, the Gateway SHALL reject the request with HTTP `409 Conflict` .
  4. If new, the Gateway SHALL execute the request and record the completed response atomically.

FR-GW-04: Boundary Schema Validation

- **Objective:** Reject malformed payloads before reaching internal domain services.
- **Specification:** The Gateway SHALL validate incoming JSON bodies against AJV JSON schemas for all registration, login, order creation, and payment payloads.
- **Error Behavior:** Invalid requests SHALL be rejected with HTTP `400 Bad Request` containing an `errors[]` array listing all failing properties (`field` , `message` , `keyword`).

FR-GW-05: Circuit Breaking & Outbound Resilience

- **Objective:** Prevent cascading service failures when an upstream microservice becomes unresponsive.
- **Specification:** The Gateway SHALL track downstream response failures. If a service accumulates 5 consecutive faults within 15 seconds, the circuit breaker SHALL trip to `OPEN` state for 15,000ms.
- **Response during OPEN:** Requests to that service SHALL fail fast with HTTP `503 Service Unavailable` and header `Retry-After: 15` . Safe requests (`GET` , `HEAD`) SHALL execute up to 2 automatic retries with exponential backoff before failing.

FR-GW-06: Unified GraphQL Facade

- **Objective:** Provide a consolidated data query endpoint for complex catalog browsing.
- **Specification:** The Gateway SHALL expose `POST /graphql` executing against an internal GraphQL schema allowing clients to filter products, retrieve category hierarchies, and execute cursor-paginated catalog queries.

5.2 Authentication & Identity Service (`AUTH`)

FR-AUTH-01: User Registration & Credential Hashing

- **Objective:** Provision user accounts with strong cryptographic credential protections.
- **Specification:** The system SHALL accept name, unique email, password, and optional phone. Passwords MUST be hashed using `bcrypt` with a minimum of 10 salt rounds before persistence. Plaintext passwords SHALL NOT be persisted or emitted in logs.

FR-AUTH-02: Role-Based Access Isolation

- **Objective:** Ensure complete separation between public customer access and administrative tools.
- **Specification:** The service SHALL expose dedicated authentication endpoints:
- `POST /api/auth/customer-login` (Restricted to `customer` role)
- `POST /api/auth/staff-login` (Restricted to `support` , `inventory_staff` , `admin`)
- `POST /api/auth/super-admin-login` (Initiates 2-step verification challenge for `super_admin`)
- `POST /api/auth/super-admin-verify` (Validates 6-digit challenge code)

FR-AUTH-03: BFF Token Secrecy Protocol

- **Objective:** Protect bearer tokens from browser-side JavaScript extraction (XSS mitigation).
- **Specification:** The Auth Service SHALL only return the raw JWT token in the response body if the incoming request contains an authentic `X-Auth-Exchange-Secret` header supplied by the Next.js BFF server. Otherwise, it SHALL return only sanitized profile metadata.

FR-AUTH-04: Instant Session Revocation via Token Versioning

- **Objective:** Instantly invalidate issued JWTs upon password changes or administrative lockout.
- **Specification:** Each user document SHALL maintain an integer `tokenVersion`. The JWT payload MUST embed `tokenVersion`. Upon password reset or administrative session revocation, the service SHALL increment `tokenVersion`. Downstream authorization filters SHALL reject any JWT whose embedded version does not match the current database value.

FR-AUTH-05: Staff Invitation Workflow

- **Objective:** Secure onboarding of operational personnel.
- **Specification:** Super Administrators SHALL issue staff invitations (`POST /api/auth/invite-staff`) specifying target email, assigned role (`support` , `inventory_staff` , `admin`), and permission scopes. The system SHALL generate a cryptographically secure token, persist its SHA-256 hash with a 48-hour expiry, and dispatch an invitation email.

5.3 Product Catalog & Inventory Service (`PROD`)

FR-PROD-01: Catalog Search & Multi-Faceted Querying

- **Objective:** Allow customers to discover products efficiently.
- **Specification:** The service SHALL expose `GET /api/products` supporting full-text keyword search (`search=`), category filtering (`category=`), brand filtering, price boundaries (`minPrice=` , `maxPrice=`), and sorting options (`sort=price_asc|price_desc|newest`).

FR-PROD-02: Dual Pagination (Cursor & Offset)

- **Objective:** Enable efficient mobile infinite scroll and traditional page-based navigation.
- **Specification:** The service SHALL support cursor pagination (`first=20` , `after=<cursor>`). The response MUST return `pagination.nextCursor` and an RFC 5988 `Link: <...>; rel="next"` header. The service SHALL simultaneously support offset pagination (`page=` , `limit=`).

FR-PROD-03: Dynamic ETag Generation & Caching (304)

- **Objective:** Conserve bandwidth and improve catalog response times.
- **Specification:** On `GET /api/products/:id` , the service SHALL compute an MD5 hash of the entity ID, modification timestamp, price, and current stock, returning an `ETag` header. If the client sends an `If-None-Match` header matching the current ETag, the service MUST respond with HTTP `304 Not Modified` with zero response body bytes.

FR-PROD-04: Optimistic Concurrency Control (If-Match / 412)

- **Objective:** Prevent lost updates when multiple staff members edit catalog records concurrently.
- **Specification:** On mutating operations (`PUT /api/products/:id`), the client MUST supply an `If-Match` header containing the known entity ETag. If the header does not match the current calculated ETag, the service MUST reject the update with HTTP `412 Precondition Failed` .

FR-PROD-05: Audited Inventory Stock Adjustments

- **Objective:** Ensure accountability for all inventory adjustments.
- **Specification:** Inventory stock modifications MUST supply an `actorUserId` and a mandatory explanation string (`stockAdjustmentReason` ≥ 3 characters). The service SHALL record an immutable audit entry in `stockHistory[]` with timestamp, actor, old stock, new stock, and reason.

5.4 Shopping Cart Service (`CART`)

FR-CART-01: Server-Side Cart Persistence

- **Objective:** Maintain customer cart selections reliably across sessions and devices.
- **Specification:** The service SHALL maintain a persistent cart document per authenticated user (`GET /api/cart`).

FR-CART-02: Item Operations & Boundary Constraints

- **Objective:** Enable granular cart modification.
- **Specification:** The service SHALL expose endpoints to add items (`POST /api/cart/items`), update item quantities (`PUT /api/cart/items/:productId`), and remove items (`DELETE /api/cart/items/:productId`). Quantities MUST be positive integers (≥ 1).

FR-CART-03: Server-Authoritative Price Calculation

- **Objective:** Prevent client-side price tampering.
- **Specification:** The Cart Service SHALL recalculate item subtotal and cart grand total on the server side using canonical product data, ignoring any price fields submitted in client payloads.

5.5 Wishlist Service (`WISH`)

FR-WISH-01: User Wishlist Management

- **Objective:** Allow customers to curate saved items for future purchase.
- **Specification:** The service SHALL maintain saved product lists per user (`smartcart_wishlist`), supporting retrieval (`GET /api/wishlist`), additions (`POST /api/wishlist`), and removals (`DELETE /api/wishlist/:productId`).

FR-WISH-02: Move to Cart Operation

- **Objective:** Frictionless transition from saved item to active purchase.
- **Specification:** The service SHALL provide a unified move-to-cart operation (`POST /api/wishlist/:productId/move-to-cart`) that invokes the Cart Service to add the item and removes it from the wishlist atomically.

5.6 Order Orchestration & Promotions Service (`ORD`)

FR-ORD-01: Distributed Checkout Saga (`placeOrder`)

- **Objective:** Orchestrate multi-service checkout while maintaining data consistency.

- **Specification:** On `POST /api/orders`, the Order Service SHALL execute the following sequential workflow:
  1. **Empty Cart Validation:** Validate that cart contains items. If empty, immediately abort with HTTP `422 Unprocessable Entity` (`urn:smartcart:error:empty-cart`).
  2. **Inventory & Price Verification:** Query Product Service to confirm item availability and active prices.
  3. **Stock Reservation:** Request Product Service to atomically reserve inventory for each item.
  4. **Promotions Evaluation:** Apply valid coupon discount codes.
  5. **Tax & Shipping Calculation:** Compute applicable shipping charges and taxes based on shipping address.
  6. **Payment Initialization:** Invoke Payment Service (`POST /api/payments/create`) to create a payment intent.
  7. **Cart Cleanup:** Request Cart Service to clear customer cart.
  8. **Compensatory Rollback:** If any downstream step fails (e.g., payment creation error), the service SHALL release all reserved stock.
  9. **Response:** Return HTTP `201 Created` with a canonical `Location: /api/orders/:id` header.

FR-ORD-02: Order State Machine Lifecycle

- **Objective:** Enforce valid business workflow transitions.
- **Specification:** Orders SHALL advance strictly through defined status transitions:
- `PENDING` $\rightarrow$ `CONFIRMED` $\rightarrow$ `SHIPPED` $\rightarrow$ `DELIVERED`
- `PENDING` / `CONFIRMED` $\rightarrow$ `CANCELLED`
- `DELIVERED` $\rightarrow$ `RETURNED` (permitted within 30 days of delivery)
- **Error Behavior:** Any attempt to execute an invalid transition (e.g., `SHIPPED` $\rightarrow$ `CANCELLED`) SHALL be rejected with HTTP `409 Conflict`.

FR-ORD-03: Optimistic Concurrency on Order Mutations

- **Objective:** Prevent concurrent administrative overwrites during order processing.
- **Specification:** Status mutations (`PATCH /api/orders/:id/status`) and cancellations SHALL validate the client's `If-Match` ETag. Stale requests MUST be rejected with HTTP `412 Precondition Failed`.

FR-ORD-04: Coupon & Discount Promotion Engine

- **Objective:** Support marketing campaigns with enforceable constraints.
- **Specification:** The service SHALL evaluate coupon codes against active promotions (`smartcart_orders.promotions`). The engine SHALL validate expiry date, minimum order amount, and usage limits, calculating either percentage (`PERCENTAGE`) or fixed amount (`FIXED`) discounts.

5.7 Payment & Financial Reconciliation Service (`PAY`)

FR-PAY-01: Digital Payment Initialization (Razorpay)

- **Objective:** Create secure payment intents for online payment channels.
- **Specification:** For online payment methods (`UPI`, `CARD`), the service SHALL initialize a payment order via the Razorpay REST API in Indian Rupee subunits (Paise).

FR-PAY-02: Cryptographic Signature Verification

- **Objective:** Verify payment authenticity before order confirmation.
- **Specification:** Upon completion of client payment, the service SHALL verify the cryptographic HMAC-SHA256 signature (`POST /api/payments/verify/:paymentId`):
  $$\text{Expected Signature} = \text{HMAC-SHA256}(\text{order_id} + "|" + \text{payment_id}, \text{RAZORPAY_KEY_SECRET})$$

If the calculated signature does not match the received signature, the payment MUST be transitioned to `FAILED` and rejected with HTTP `400 Bad Request`.

FR-PAY-03: Asynchronous Webhook Processing

- **Objective:** Guarantee financial state synchronization even if the client disconnects.
- **Specification:** The service SHALL expose `POST /api/payments/webhook` with raw request body signature verification. It SHALL process events for `payment.captured`, `payment.failed`, and `refund.processed`.

FR-PAY-04: Cash on Delivery (COD) Reconciliation

- **Objective:** Support offline payment fulfillment.
- **Specification:** The service SHALL record COD payments with status `PENDING`, transitioning to `SUCCESS` upon courier delivery confirmation.

FR-PAY-05: Automated Financial Refunds

- **Objective:** Enable authorized refund processing.
- **Specification:** Super Administrators SHALL authorize refunds (`POST /api/payments/refund/:paymentId`). For online transactions, the service SHALL invoke the Razorpay Refund API, record the refund transaction, and update the associated order status.

5.8 Notification & Communication Service (`NOTIF`)

FR-NOTIF-01: Transactional Event Notifications

- **Objective:** Provide automated customer and staff communications.
- **Specification:** The service SHALL dispatch email notifications for key domain events:
- Account welcome and email verification.
- Password recovery reset links.
- Super Admin 2-step verification challenge codes.
- Order confirmation with itemized summary.
- Shipping dispatch with courier tracking link.
- Return evaluation decisions and refund confirmation.

FR-NOTIF-02: Provider Decoupling & Test Fallback

- **Objective:** Prevent external API dependencies from blocking development and automated testing.
- **Specification:** The service SHALL integrate with the **Resend API**. If `RESEND_API_KEY` is not provided, the service SHALL automatically route email payloads to a structured mock console logger without failing the transaction.

5.9 Storefront & Management Web Client (`UI`)

FR-UI-01: Persona-Specific User Portals

- **Objective:** Deliver tailored user experiences based on authenticated roles.
- **Specification:** The Next.js 15 application SHALL render dedicated interfaces:
- **Storefront:** Dynamic product catalog, interactive cart drawer, multi-step checkout, and order confirmation.
- **Customer Portal:** Order history, shipment tracking status, and 30-day return submission forms.
- **Staff Portal:** Catalog inventory adjustments with mandatory reason modal, return review workflow.

- **Admin Suite:** Order dispatch control, carrier tracking assignment, coupon creator, review moderation.
- **Super Admin Console:** Staff account invitation, refund authorizations, store configuration.

FR-UI-02: Resilient Client-Side Data Querying

- **Objective:** Ensure smooth, responsive user interactions.
- **Specification:** Client-side data fetching SHALL be managed via `@tanstack/react-query` implementing optimistic UI updates, background cache revalidation, and graceful error boundaries that render human-friendly messages from RFC 7807 Problem Details.

---

6. Non-Functional Requirements (NFRs)

6.1 Security & Cryptographic Specifications

- **NFR-SEC-01 (Password Hashing):** All user passwords SHALL be hashed using `bcrypt` with a work factor ≥ 10 prior to persistence.
- **NFR-SEC-02 (Token Confidentiality):** JWTs SHALL be signed using HMAC-SHA256 with secret keys containing ≥ 256 bits of entropy. Tokens SHALL be delivered to browsers exclusively via `HttpOnly`, `SameSite=Lax`, and `Secure` cookies.
- **NFR-SEC-03 (Timing Attack Prevention):** Validation of internal service secrets and webhook signatures SHALL utilize constant-time comparison (`crypto.timingSafeEqual`).
- **NFR-SEC-04 (HTTP Header Hardening):** The API Gateway SHALL enforce security headers via `helmet`: `X-Content-Type-Options: nosniff`, `X-Frame-Options: SAMEORIGIN`, and disabled `X-Powered-By`.
- **NFR-SEC-05 (CORS Restrictions):** Cross-Origin Resource Sharing (CORS) SHALL strictly enforce origin allowlists; wildcard `*` with credentials SHALL NOT be permitted.

6.2 Reliability, Fault Tolerance & Resilience

- **NFR-REL-01 (Circuit Breaker Tripping):** Upstream failures exceeding 5 faults in 15 seconds SHALL trip the Gateway circuit breaker to `OPEN` state, returning HTTP `503 Service Unavailable` with `Retry-After: 15`.
- **NFR-REL-02 (Safe Retry Semantics):** Automated retries with exponential backoff SHALL be restricted strictly to idempotent HTTP methods (`GET`, `HEAD`). Mutating requests (`POST`, `PUT`, `DELETE`) SHALL NEVER be retried without idempotency keys.
- **NFR-REL-03 (Idempotency Retention):** Processed idempotency records in MySQL SHALL be retained for a minimum of 24 hours.

6.3 Performance & Caching Attributes

- **NFR-PERF-01 (Gateway Overhead Budget):** Gateway reverse-proxy overhead SHALL NOT exceed 15ms at the 95th percentile under standard load.
- **NFR-PERF-02 (Bandwidth Conservation):** Unmodified resources requested with matching `If-None-Match` headers SHALL respond with HTTP `304 Not Modified` and zero-length body.
- **NFR-PERF-03 (Payload Size Limits):** Incoming JSON request body sizes SHALL be restricted to 200KB at the Gateway and 100KB at downstream microservices to mitigate DoS vulnerabilities.

6.4 Observability, Logging & Auditability

- **NFR-OBS-01 (Distributed Correlation):** The Gateway SHALL inject a unique `X-Request-Id` UUID into every request. All downstream services SHALL include this ID in every log entry and internal HTTP call.

- **NFR-OBS-02 (Structured JSON Logs):** All server logs SHALL be emitted as single-line JSON strings containing `timestamp`, `level`, `service`, `requestId`, `method`, `path`, `status`, and `durationMs`.
- **NFR-OBS-03 (Immutable Audit Trails):** Security-sensitive mutations (role adjustments, inventory overrides, refund approvals) SHALL be written to immutable audit collections.

6.5 Maintainability, Portability & Configuration Standards

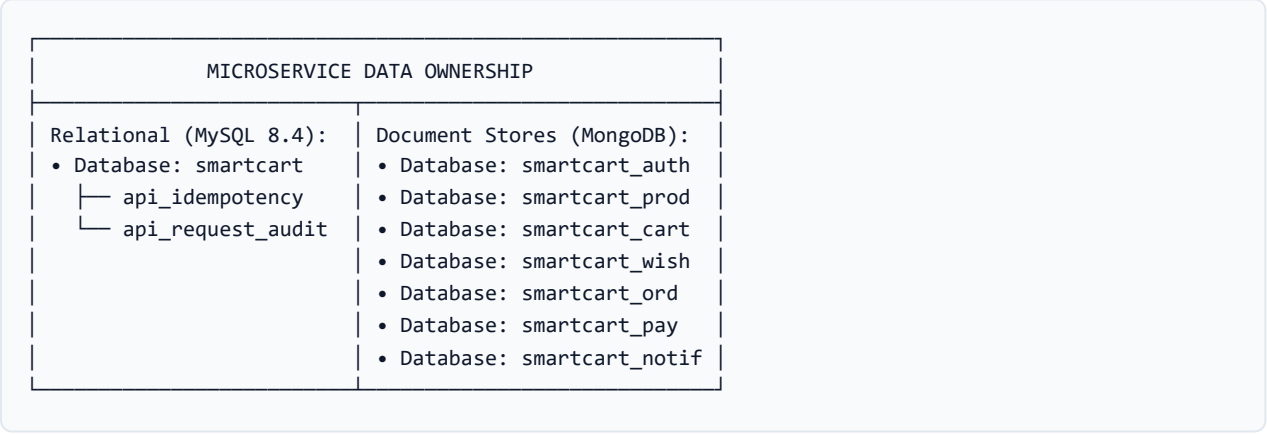
- **NFR-MAINT-01 (Uniform Problem Details):** All non-2xx HTTP responses across all services SHALL conform to RFC 7807/9457 `application/problem+json`.
- **NFR-MAINT-02 (Twelve-Factor Configuration):** All service port allocations, database URIs, timeouts, and cryptographic secrets SHALL be configurable exclusively via environment variables.

---

7. Data Requirements & Persistence Models

7.1 Database Isolation & Data Ownership Rules

Each microservice owns an isolated persistence boundary. No cross-database queries or shared tables are permitted.



7.2 Relational Data Schema (MySQL `smartcart`)

Table: `api_idempotency`

Stores durable request/response states for idempotent mutations.

```
CREATE TABLE IF NOT EXISTS api_idempotency (
    idempotency_key VARCHAR(255) PRIMARY KEY,
    route_path VARCHAR(255) NOT NULL,
    method VARCHAR(16) NOT NULL,
    request_hash CHAR(64) NOT NULL,
    status_code INT NOT NULL,
    response_headers JSON NOT NULL,
    response_body LONGTEXT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    INDEX idx_created_at (created_at)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Table: `api_request_audit`

Provides immutable edge request audit logs at the API Gateway.

```
CREATE TABLE IF NOT EXISTS api_request_audit (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    request_id VARCHAR(64) NOT NULL,
    route_path VARCHAR(255) NOT NULL,
    method VARCHAR(16) NOT NULL,
    status_code INT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    INDEX idx_request_id (request_id),
    INDEX idx_created_at (created_at)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

7.3 Document Data Schemas (MongoDB)

1. Identity Domain (`smartcart_auth`)

- **`users` Collection:**
- `_id` : ObjectId
- `name` : String, required
- `email` : String, required, unique, indexed
- `password` : String, required, select: false (Bcrypt hash)
- `role` : String, enum: `['customer', 'support', 'inventory_staff', 'admin', 'super_admin']`
- `tokenVersion` : Number, default 0
- `isActive` : Boolean, default true
- `addresses` : Array of Address subdocuments
- `passwordResetTokenHash` : String (SHA-256)
- `staffInviteTokenHash` : String (SHA-256)
- **`authauditevents` Collection:**
- `actorUserId` , `action` , `targetUserId` , `outcome` , `requestId` , `createdAt`

2. Product Catalog Domain (`smartcart_products`)

- **`products` Collection:**
- `_id` : ObjectId
- `name` : String, required, text-indexed
- `description` : String
- `price` : Number, required, min: 0
- `category` : String, required, indexed
- `brand` : String
- `stock` : Number, required, min: 0
- `images` : Array of URL strings
- `stockHistory` : Array of `{ actorUserId, previousStock, newStock, reason, changedAt }`
- `ratingAverage` : Number, default: 0
- `archivedAt` : Date, default: null
- **`reviews` Collection:**

- `productId`, `userId`, `rating` (1–5), `comment`, `isModerated`, `createdAt`

3. Shopping Cart Domain (`smartcart_cart`)

- **`carts` Collection:**
- `userId`: String, required, unique, indexed
- `items`: Array of `{ productId, name, price, quantity, imageUrl }`
- `subtotal`: Number, default: 0
- `updatedAt`: Date

4. Wishlist Domain (`smartcart_wishlist`)

- **`wishlists` Collection:**
- `userId`: String, required, unique, indexed
- `productIds`: Array of String/ObjectId
- `updatedAt`: Date

5. Order & Promotion Domain (`smartcart_orders`)

- **`orders` Collection:**
- `_id`: ObjectId
- `userId`: String, required, indexed
- `items`: Array of OrderItem subdocuments
- `subtotal`: Number, required
- `discountAmount`: Number, default: 0
- `shippingFee`: Number, default: 0
- `taxAmount`: Number, default: 0
- `totalAmount`: Number, required
- `couponCode`: String
- `shippingAddress`: ShippingAddress subdocument
- `orderStatus`: Enum: `['PENDING', 'CONFIRMED', 'SHIPPED', 'DELIVERED', 'CANCELLED', 'RETURNED']`
- `paymentStatus`: Enum: `['PENDING', 'SUCCESS', 'FAILED', 'REFUNDED']`
- `paymentMethod`: Enum: `['UPI', 'CARD', 'COD']`
- `trackingNumber`: String
- `carrier`: String
- `returnReason`: String
- `createdAt`, `updatedAt`: Timestamps
- **`promotions` Collection:**
- `code`: String, required, unique, uppercase
- `discountType`: Enum: `['PERCENTAGE', 'FIXED']`
- `discountValue`: Number, required
- `minOrderAmount`: Number, default: 0
- `expiresAt`: Date
- `isActive`: Boolean

6. Payment & Reconciliation Domain (`smartcart_payments`)

- **`payments` Collection:**
- `orderId` : String, required, unique, indexed
- `userId` : String, required, indexed
- `amount` : Number, required (Rupees)
- `paymentMethod` : Enum: `['UPI', 'CARD', 'COD']`
- `status` : Enum: `['PENDING', 'SUCCESS', 'FAILED', 'REFUNDED']`
- `razorpayOrderId` : String
- `razorpayPaymentId` : String
- `razorpaySignature` : String
- `events` : Array of `{ type, message, timestamp }`
- `refunds` : Array of `{ refundId, amount, status, reason, createdAt }`

7. Notification Domain (`smartcart_notifications`)

- **`notifications` Collection:**
- `recipientEmail` : String, required
- `templateId` : String, required
- `payload` : Object
- `status` : Enum: `['QUEUED', 'SENT', 'FAILED']`
- `createdAt` : Date

7.4 Data Integrity, Optimistic Concurrency & Lifecycle Rules

1. **Optimistic Concurrency Control:** Products and Orders documents shall maintain state versions. Modifications shall verify client `If-Match` ETags prior to saving to prevent lost updates.
2. **Soft Deletions:** Products shall not be deleted physically; instead, an `archivedAt` timestamp shall be assigned.
3. **Audit Immutability:** Stock adjustment logs and payment audit collections shall strictly forbid `UPDATE` and `DELETE` operations.

---

8. Verification, Validation & Acceptance Criteria

8.1 Verification Methodology

Every requirement in this specification shall be verified prior to system delivery using one or more of the standard verification methods:
- **[I] Inspection:** Visual examination of documentation, code structure, or schemas.
- **[D] Demonstration:** Operational observation of system behavior under standard conditions.
- **[T] Test:** Execution of automated unit, integration, or end-to-end test suites.
- **[A] Analysis:** Mathematical or logical evaluation of performance benchmarks and security proofs.

8.2 Requirements Traceability Matrix (RTM)

The following matrix maps each engineering milestone (Assignments 1 through 6) to the governing functional/non-functional requirements, verification methods, and acceptance criteria:

Engineering Milestone	Syllabus / Project Objective	Governing Requirements	Verification Method	Formal Acceptance Criteria
Milestone 1 *(Foundations)*	Raw HTTP & Project Architecture	`FR-GW-01` `NFR-MAINT-02`	**[I], [T]**	Clean service directory layout; `.env.example` configurations; functional raw HTTP handling across all services.
Milestone 2 *(Services & Contracts)*	Service Boundaries, Schema Isolation, Checkout Saga	`FR-ORD-01` `FR-PROD-01` `FR-CART-01`	**[T], [D]**	8 microservices operational on allocated ports; strict database isolation; `placeOrder` saga coordinates stock reservation and compensatory release.
Milestone 3 *(Partner Integration)*	External Payment Edge, Webhooks, Signature Check	`FR-PAY-01` `FR-PAY-02` `FR-PAY-03`	**[T], [D]**	Razorpay order creation in Paise; HMAC-SHA256 signature verification passes; invalid signatures transition to `FAILED`; webhook endpoint processes callbacks.
Milestone 4 *(REST & Resilience)*	REST Naming, OpenAPI Spec, Idempotency, Circuit Breakers	`FR-GW-03` `FR-GW-05` `NFR-REL-01`	**[T], [A]**	Plural resource URIs; OpenAPI 3.0 specification available at `/docs`; duplicate `Idempotency-Key` replays cached response with header; circuit breaker opens on 5 faults.
Milestone 5 *(HTTP Semantics)*	ETags, Concurrency, Content Negotiation, Rate Limits	`FR-PROD-03` `FR-PROD-04` `FR-GW-02` `FR-GW-06`	**[T]**	`If-None-Match` returns 304; stale `If-Match` returns 412; unsupported `Accept` returns 406; `OPTIONS` returns 204 with `Allow`; rate limiter emits 429 with `Retry-After`.
Milestone 6 *(Robustness & Errors)*	Honest HTTP Codes, RFC 7807 Errors, Multi-Field Validation	`NFR-MAINT-01` `FR-GW-04` `FR-ORD-01`	**[T], [I]**	15+ honest HTTP status codes; all errors return `application/problem+json`; 201 Created emits `Location`; empty cart checkout returns 422; 500 errors safely masked.

8.3 Standard HTTP Status Code Baseline

The system shall strictly emit honest, standardized HTTP status codes in accordance with RFC 7231:

HTTP Status	Standard Reason Phrase	System Behavioral Context & Trigger
`200`	OK	Successful `GET` query, or idempotent modification completion.
`201`	Created	Resource successfully created (`POST /api/orders` , `/api/products`); MUST emit `Location` header.
`204`	No Content	Preflight or explicit `OPTIONS` request; MUST emit `Allow` header listing supported HTTP methods.
`304`	Not Modified	Conditional `GET` where request `If-None-Match` matches entity ETag; body MUST be zero bytes.

HTTP Status	Standard Reason Phrase	System Behavioral Context & Trigger
`400`	Bad Request	AJV schema validation failure or invalid payment signature; returns Problem Details with `errors[]`.
`401`	Unauthorized	Missing or expired JWT bearer token on protected endpoint.
`403`	Forbidden	Authenticated user lacks necessary RBAC permission claim for requested action.
`404`	Not Found	Requested URI route or target entity identifier does not exist.
`406`	Not Acceptable	Incoming `Accept` header demands an unsupported media format (neither JSON nor XML).
`409`	Conflict	Duplicate resource identifier, illegal order state machine transition, or reused `Idempotency-Key` with altered payload.
`412`	Precondition Failed	Optimistic concurrency mismatch: request `If-Match` does not match resource's current ETag.
`422`	Unprocessable Entity	Semantically invalid domain payload (e.g., checkout initiated with an empty items array).
`429`	Too Many Requests	Rate limit quota exceeded; MUST emit `Retry-After: <seconds>` header.
`500`	Internal Server Error	Uncaught server exception; stack traces MUST be masked from client responses.
`503`	Service Unavailable	Downstream microservice offline or circuit breaker tripped to `OPEN`; MUST emit `Retry-After`.
`504`	Gateway Timeout	Downstream microservice call exceeded `AbortSignal.timeout` threshold (5000ms).

8.4 RFC 7807 Problem Details Error Catalogue

All error responses across all platform microservices shall strictly conform to the following standardized catalogue:

Error Type URI Slug	Status	Standard Title	Detailed Domain Condition
`urn:smartcart:error:empty-cart`	`422`	Unprocessable Entity	Checkout initiated with an empty items array.
`urn:smartcart:error:item-unavailable`	`409`	Conflict	Requested product is out of stock or archived.
`urn:smartcart:error:payment-declined`	`400`	Bad Request	Payment authorization declined or signature invalid.
`urn:smartcart:error:order-not-found`	`404`	Not Found	Target order identifier does not exist.
`urn:smartcart:error:product-not-found`	`404`	Not Found	Target product identifier does not exist.
`urn:smartcart:error:illegal-transition`	`409`	Conflict	Attempted invalid order lifecycle transition.

Error Type URI Slug	Status	Standard Title	Detailed Domain Condition
`urn:smartcart:error:precondition-failed`	`412`	Precondition Failed	Concurrency conflict: entity modified by another actor.
`urn:smartcart:error:rate-limit-exceeded`	`429`	Too Many Requests	Request quota exhausted within active window.
`urn:smartcart:error:not-acceptable`	`406`	Not Acceptable	Requested `Accept` media type is unsupported.
`urn:smartcart:error:validation-error`	`400`	Bad Request	Payload failed AJV schema constraints.
`urn:smartcart:error:authentication-required`	`401`	Unauthorized	Bearer token missing or malformed on protected route.
`urn:smartcart:error:insufficient-permissions`	`403`	Forbidden	User role lacks permission claim for target operation.
`urn:smartcart:error:service-unavailable`	`503`	Service Unavailable	Circuit breaker tripped open or upstream down.

---

9. Appendix & Sign-Off

9.1 Glossary & Acronyms

- **AJV:** Another JSON Schema Validator — high-performance schema validator for JavaScript/Node.js.
- **BFF:** Backend-for-Frontend — architectural pattern where a dedicated backend server acts as an interface layer between client browsers and internal microservices.
- **COD:** Cash on Delivery — payment method where financial settlement occurs upon physical package handover.
- **ETag:** Entity Tag — HTTP response header used for web cache validation and conditional requests.
- **HMAC:** Hash-Based Message Authentication Code — cryptographic construction using a secret key and a hash function (SHA-256).
- **JWT:** JSON Web Token — compact, URL-safe means of representing claims between two parties.
- **NFR:** Non-Functional Requirement — system quality attribute specifying operational criteria rather than specific behaviors.
- **OCC:** Optimistic Concurrency Control — concurrency control method assuming multiple transactions can complete without interfering, verifying state version prior to commit.
- **RBAC:** Role-Based Access Control — policy-neutral access control mechanism defined around roles and privileges.
- **REST:** Representational State Transfer — architectural style for distributed hypermedia systems.
- **RFC:** Request for Comments — formal standards documents drafted by the Internet Engineering Task Force (IETF).
- **SDLC:** Software Development Life Cycle — structured process used to design, develop, test, and maintain high-quality software.
- **SRS:** Software Requirements Specification — formal specification document detailing system requirements prior to implementation.

9.2 Engineering Sign-Off Record

Role	Name	Title / Organization	Status	Date
Lead Architect	Group 03 Engineering Lead	SmartCart Core Team	**APPROVED**	2026-10-16
Backend Lead	Group 03 Microservices Lead	SmartCart Core Team	**APPROVED**	2026-10-16
Frontend Lead	Group 03 Web Systems Lead	SmartCart Core Team	**APPROVED**	2026-10-16
QA & Verification	Group 03 Test & Quality Lead	SmartCart Core Team	**APPROVED**	2026-10-16

End of Software Requirements Specification (SRS) — SmartCart Distributed E-Commerce Platform